How Inwire works, end to end

Twelve diagrams, each a set of cards joined by labelled arrows. A card is a process, a module or a file; an arrow is something that moves, and its label says what. Colour says which process a card belongs to, and the colours are the same on every page.

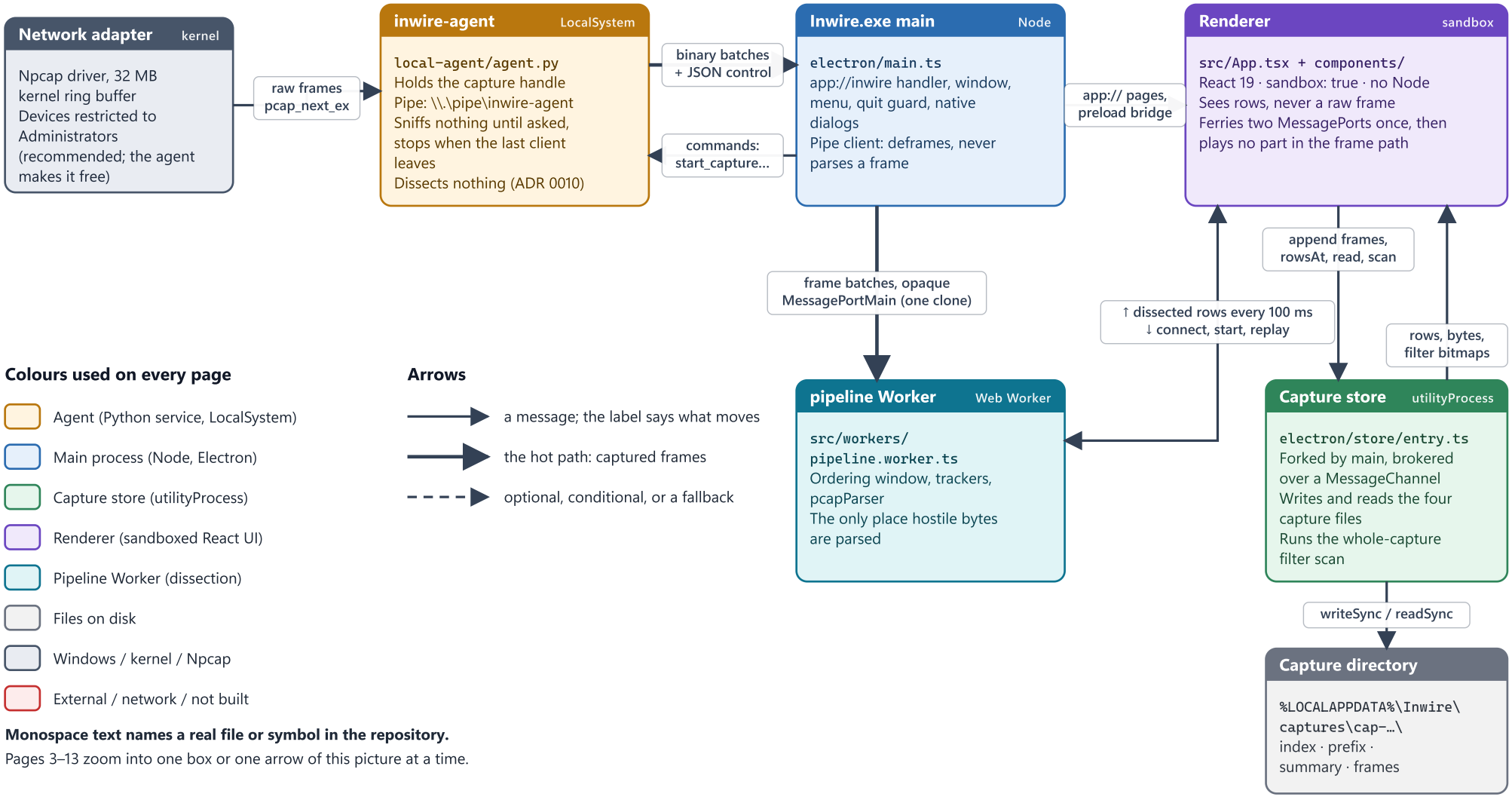
Drawn from the repository as of 2026-09-18: CLAUDE.md, the ADRs, and the source tree. Where a card names a file in monospace, that file exists and holds the decision.

- Agent (Python service, LocalSystem)
- Capture store (utilityProcess)
- Pipeline Worker (dissection)
- Windows / kernel / Npcap
- Main process (Node, Electron)
- Renderer (sandboxed React UI)
- Files on disk
- External / network / not built

CONTENTS

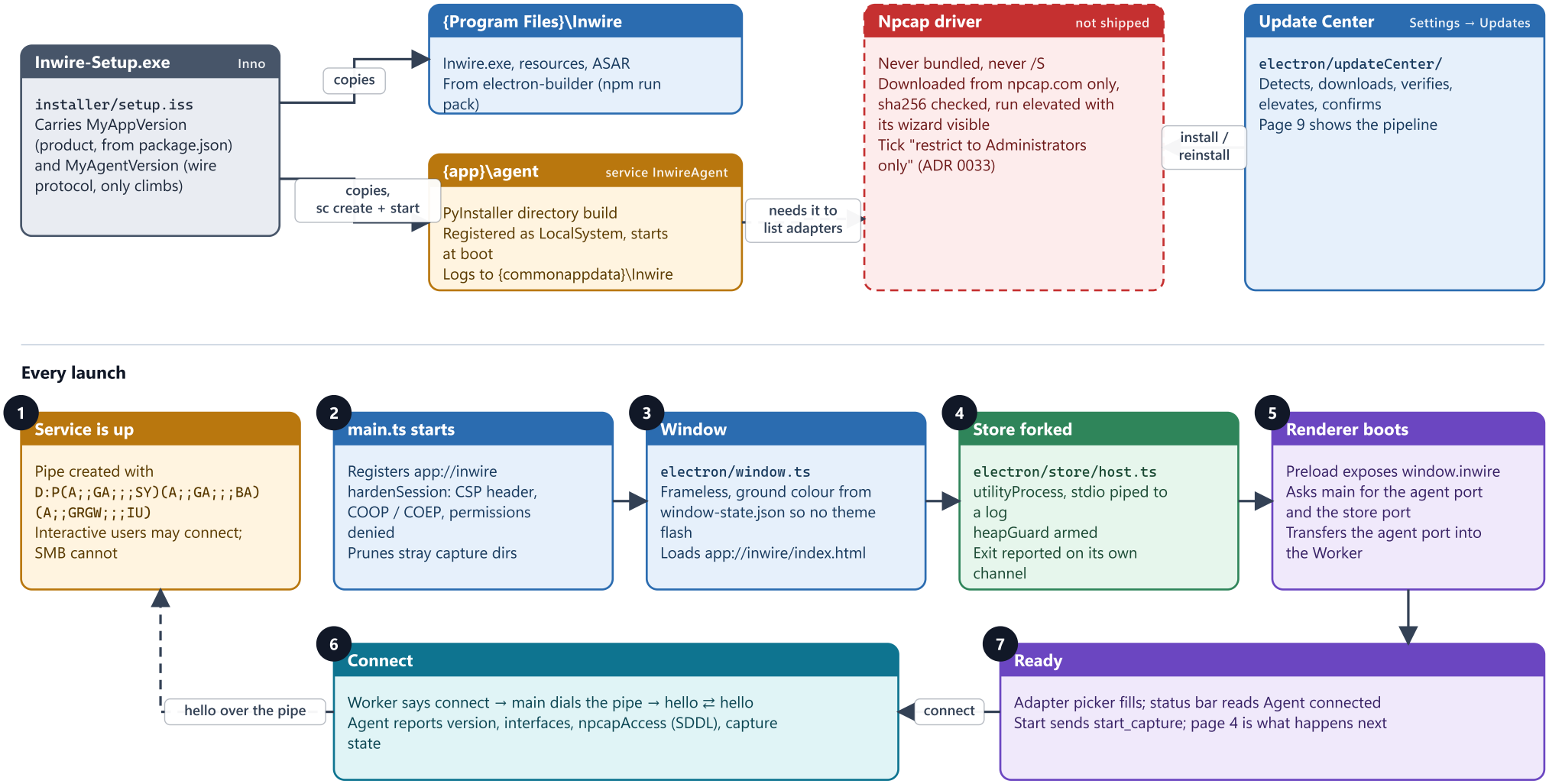
1	Five processes, one direction of flow	2
2	One installer, two destinations, and what happens on every launch	3
3	From the wire to a row on screen, in nine hops	4
4	Summary on arrival, tree on demand	5
5	A bounded RAM window in front of an unbounded file on disk	6
6	One expression, one evaluator, one answer over the whole capture	7
7	A frameless shell in a C shape, and views that drill down	8
8	The only place the app changes anything on the machine	9
9	Who may touch what	10
10	Plain tsx for everything that can be, a real Electron for what cannot	11
11	Import streams to disk, export goes through a native dialog	12
12	The map from concept to file	13

Five processes, one direction of flow



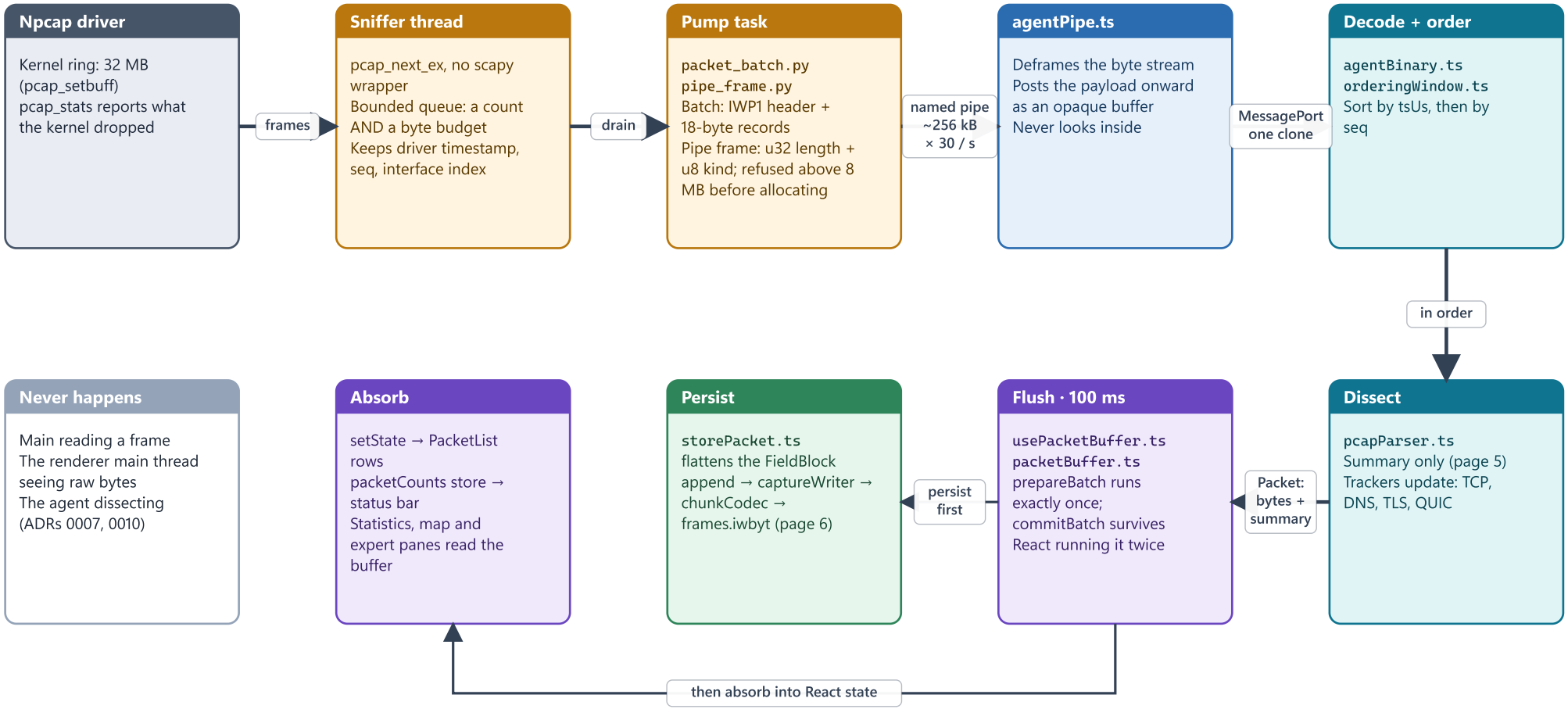
Frames move left to right and down: the kernel driver hands them to a privileged Python service, which pushes them over a named pipe to the Electron main process, which forwards the bytes untouched to a Web Worker that dissects them. The React UI only ever sees dissected rows, and it hands every frame to a separate store process that owns the files on disk. Nothing in this picture is a server: every box runs on this machine.

One installer, two destinations, and what happens on every launch



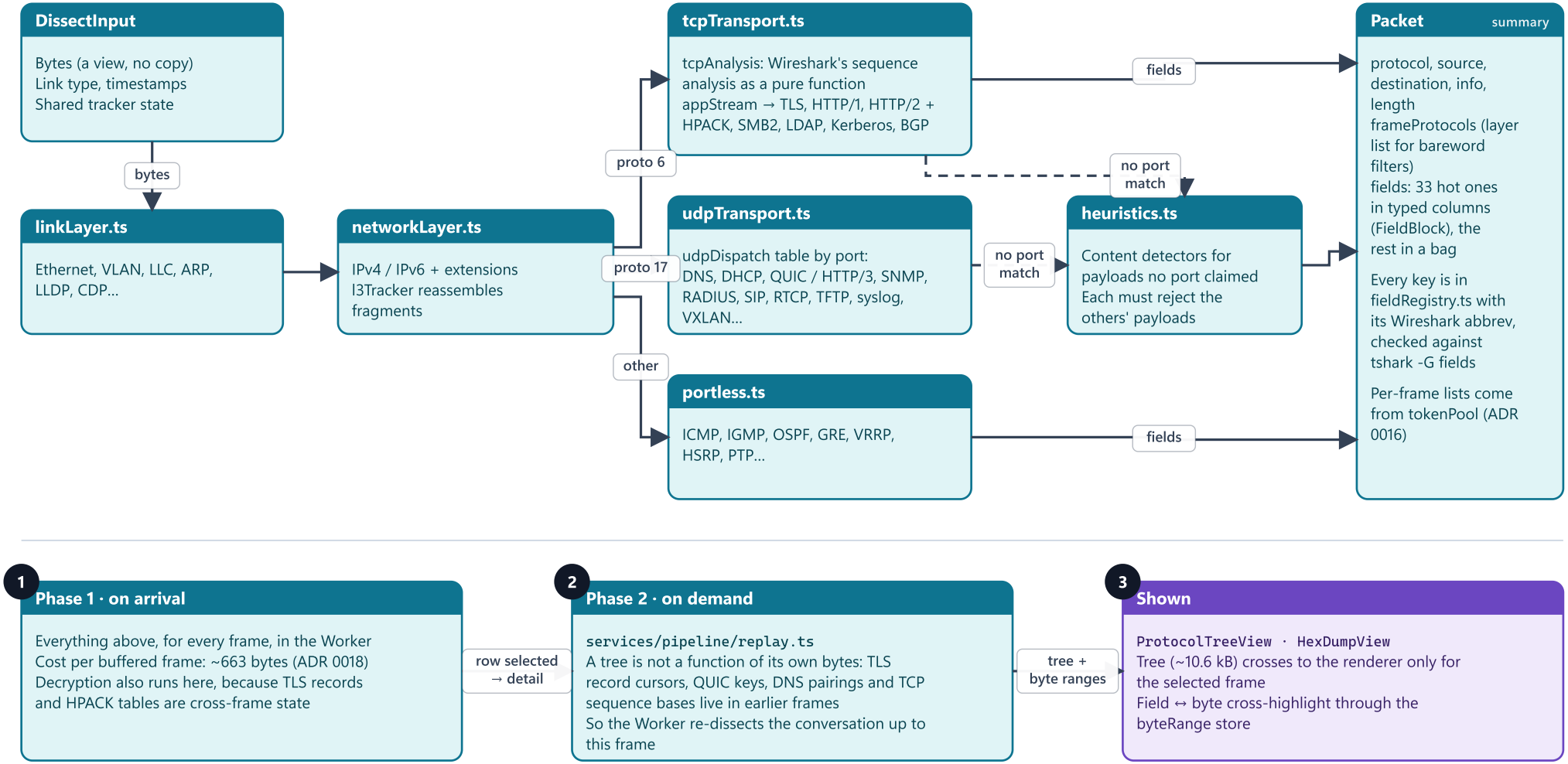
Top: the Inno Setup installer copies the app and the agent and registers the agent as a boot-start Windows service. It deliberately does not install Npcap: the free Npcap licence forbids silent installs, so the driver is fetched later from npcap.com through the Update Center and installed with its own wizard on screen (ADR 0032). Bottom: the numbered sequence a launch goes through, from the service that is already running to the first hello on the pipe.

From the wire to a row on screen, in nine hops



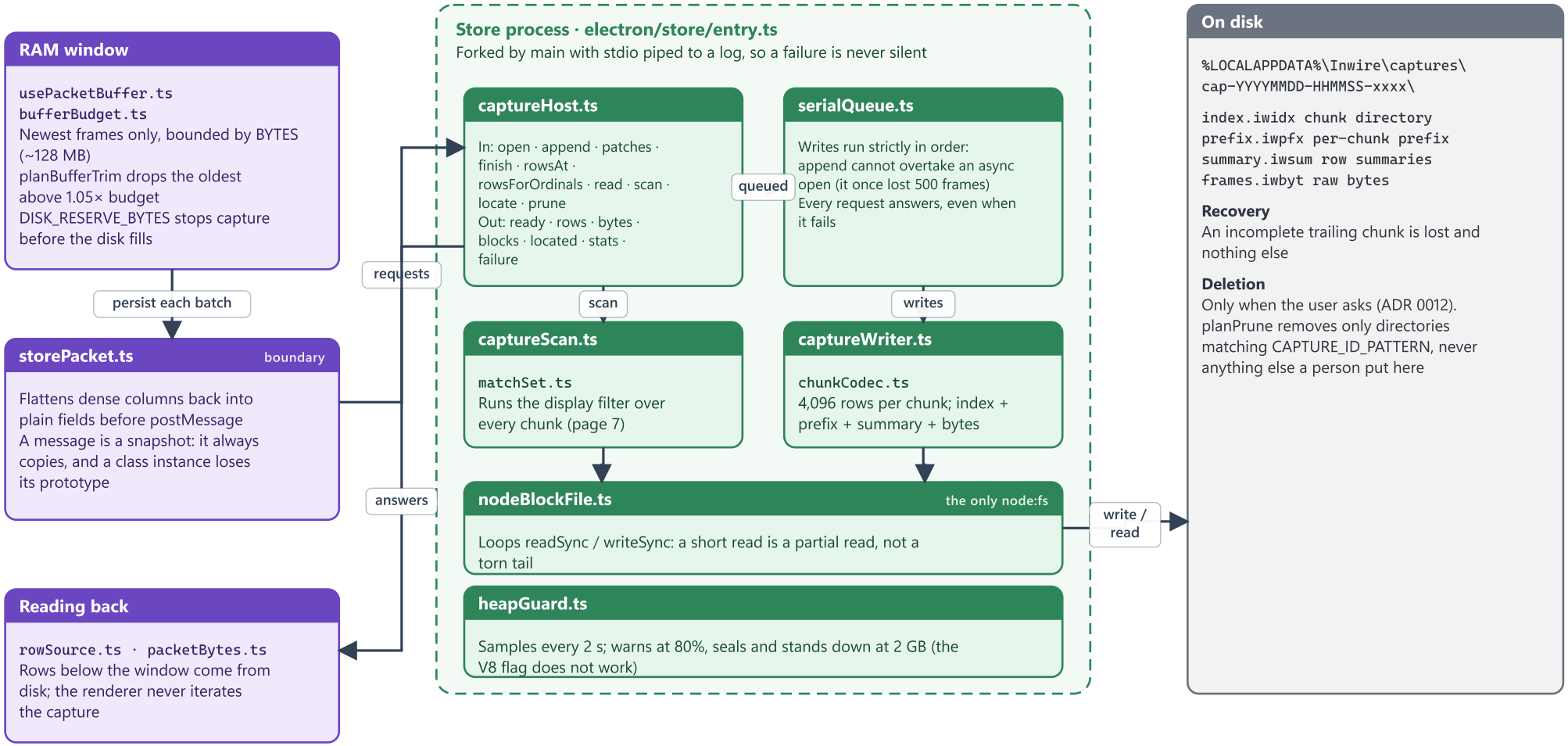
Read the top row left to right and the bottom row right to left. The first three hops are the agent; from the pipe onward the bytes are copied exactly once (MessagePortMain clones an ArrayBuffer, it cannot transfer one) and are parsed only inside the Worker. The Worker emits summaries, not trees, on a 100 ms timer. The renderer persists each batch to the store before it absorbs it into React state, so the disk copy is complete even if the RAM window later trims.

Summary on arrival, tree on demand



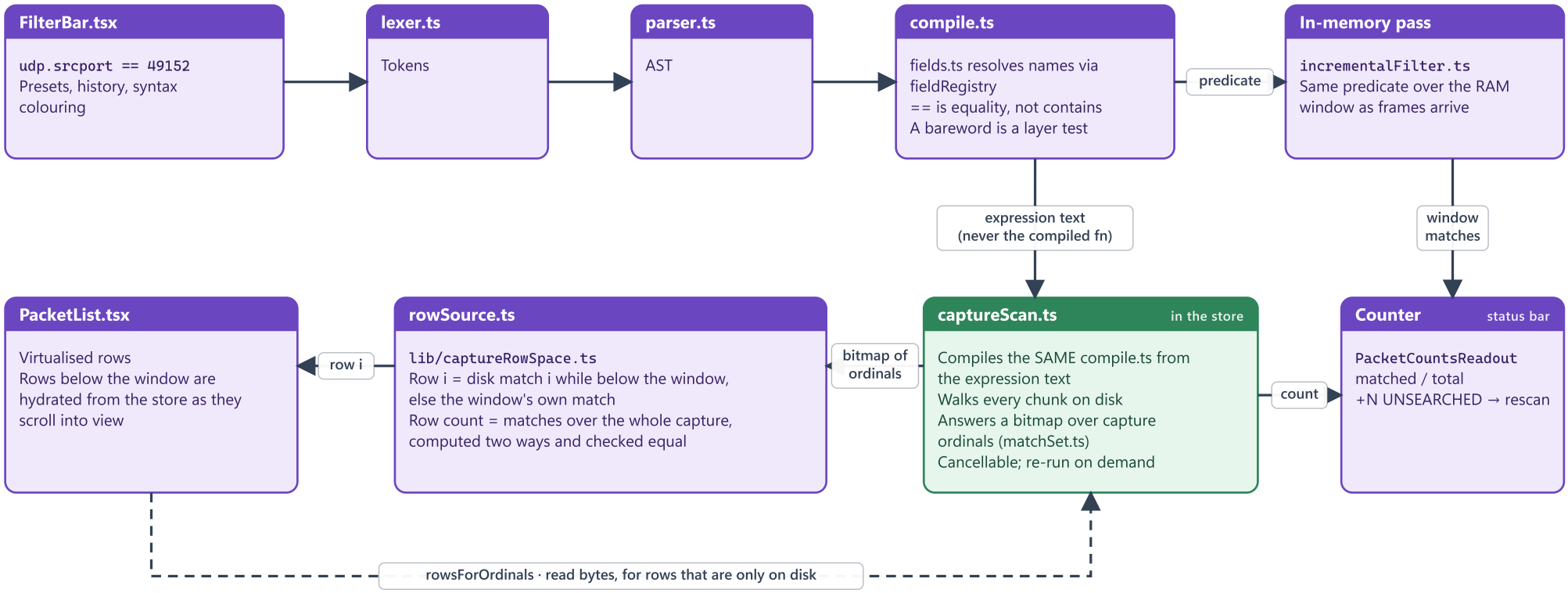
Top: the decomposed parser (ADR 0023). The link layer is read, not assumed; fragments are reassembled; TCP and UDP fan out to leaf protocols by port, and heuristics catch payloads no port claimed. Every field is registered with its Wireshark name, so a filter learnt in Wireshark works here. Bottom: a full tree is never computed up front. It depends on state built by earlier frames, so it is produced by replaying the frame's conversation when a row is selected.

A bounded RAM window in front of an unbounded file on disk



The renderer keeps only the newest frames, bounded by bytes rather than by count because a frame is anywhere from 60 bytes to 64 kB. Everything goes to the store process first, which writes 4,096-row chunks into four files. The flatten at the boundary exists because a structured clone drops class prototypes: for months the store silently stored nothing because every frame arrived without its read method (CLAUDE.md §8). Crossing the boundary costs 16% of the write; measured, not assumed (ADR 0028).

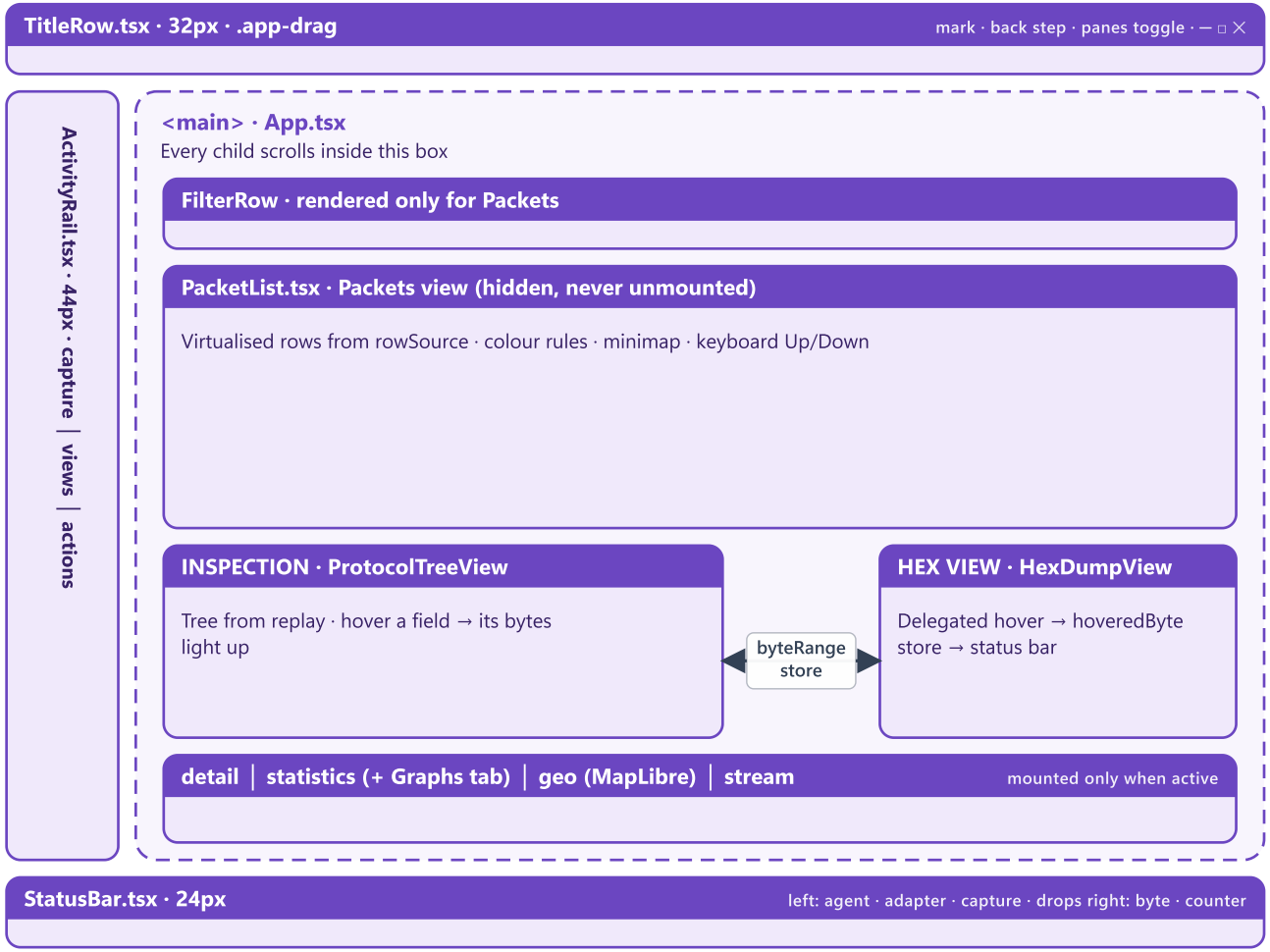
One expression, one evaluator, one answer over the whole capture



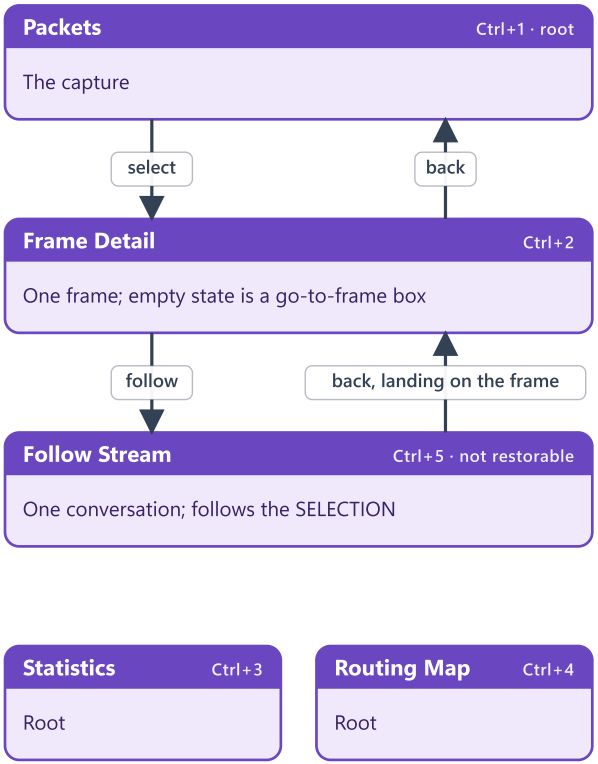
Why the arithmetic is tested
scripts/capture/store-protocol.ts runs the scan and the in-memory filter over the same frames and compares ordinals; scripts/session/capture-row-space.ts checks that the list has exactly as many rows as the filter has matches. Getting row i wrong shows the wrong frame at the right height, which nobody notices.

The filter text is tokenised, parsed and compiled in the renderer, and the same compile.ts runs inside the store over every chunk on disk. There is deliberately no second, faster evaluator: it would disagree with the in-memory filter on exactly the frames nobody can see to check (ADR 0031). The list is assembled from two sources: matched ordinals below the RAM window are fetched from disk, matches inside it come from memory. The counter shows matched / total for the whole capture and offers a rescan when frames arrived after the scan.

A frameless shell in a C shape, and views that drill down



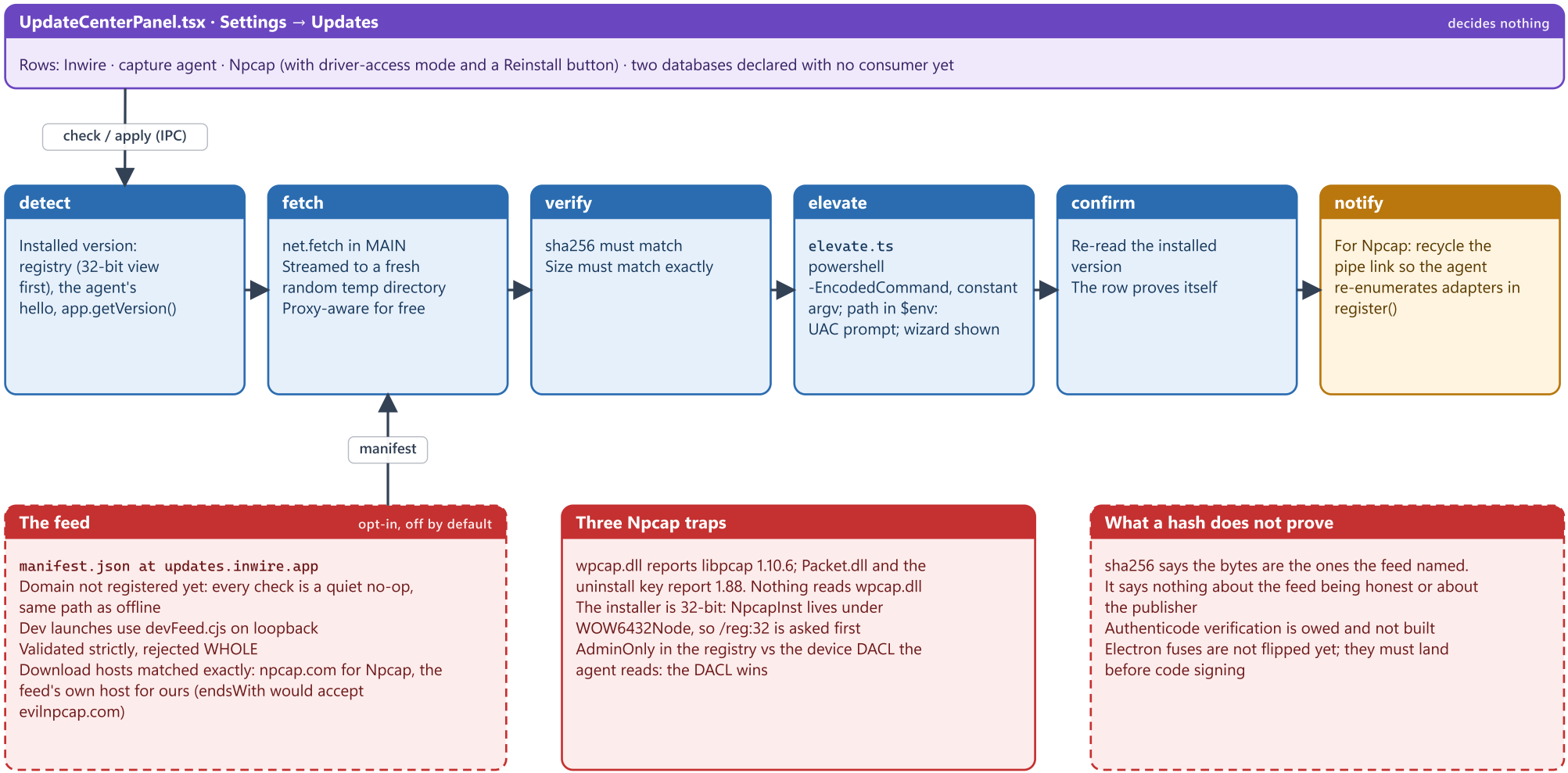
Views and their back steps



backTargetFor in viewPrefs.ts is a TOTAL record: a new view cannot be added without deciding whether it is a root. Escape: any view → Packets, one hop.

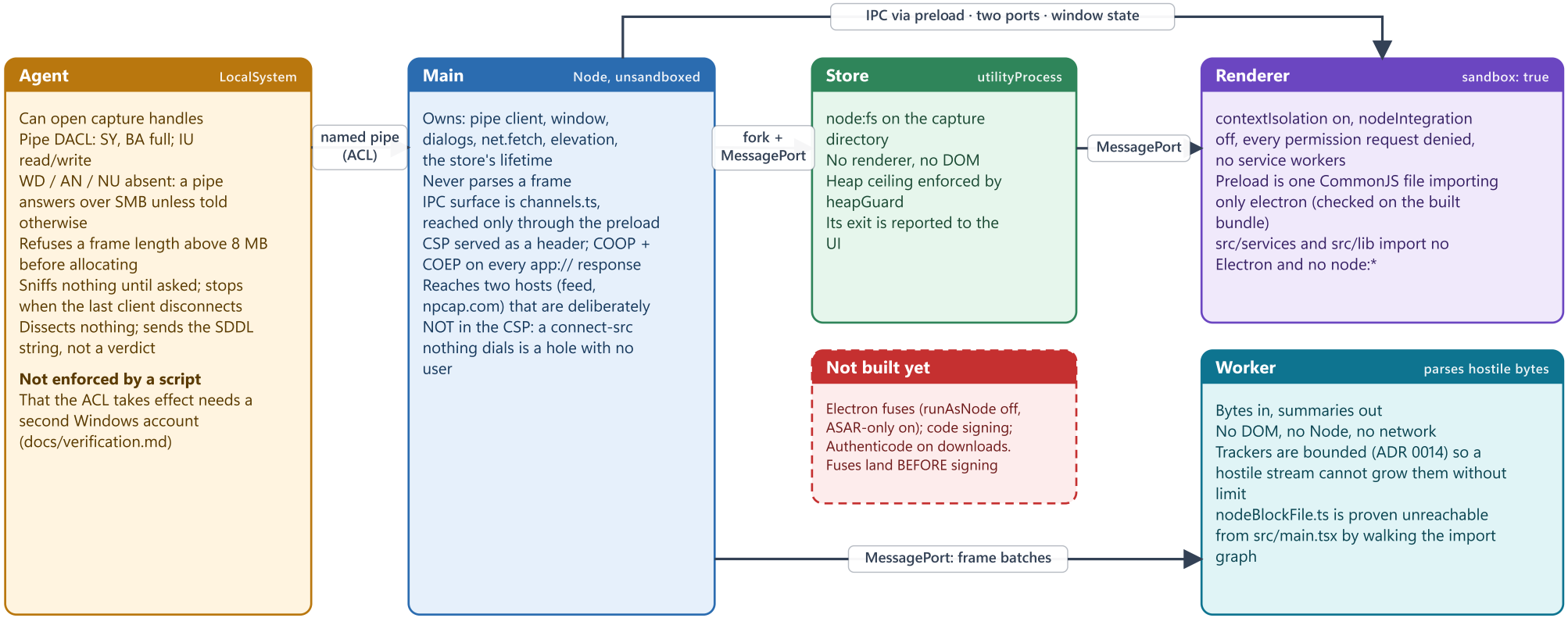
Left: the shell. Electron draws no frame; the title row carries the app's own caption buttons and is the only drag region, the rail and the workspace are two floating cards on a 4px gutter, and the status bar is a readout with one control (the rescan). Right: views drill down. Packets is the capture, Frame Detail is one frame of it, Follow Stream is one conversation that frame belongs to; each has a back step. Statistics and the Routing Map are roots reached from the rail.

The only place the app changes anything on the machine



Settings → Updates lists the app, the agent and the Npcap driver. The renderer arranges rows and decides nothing; every step runs in main. The download happens in main so the renderer never learns the path that is about to run elevated; the elevated command line is a constant, with the path travelling in the environment so a space cannot become a second argument. A declined UAC prompt is "cancelled", never "failed". The feed is the one remote input that can nominate an executable, so it is validated strictly and rejected whole.

Who may touch what

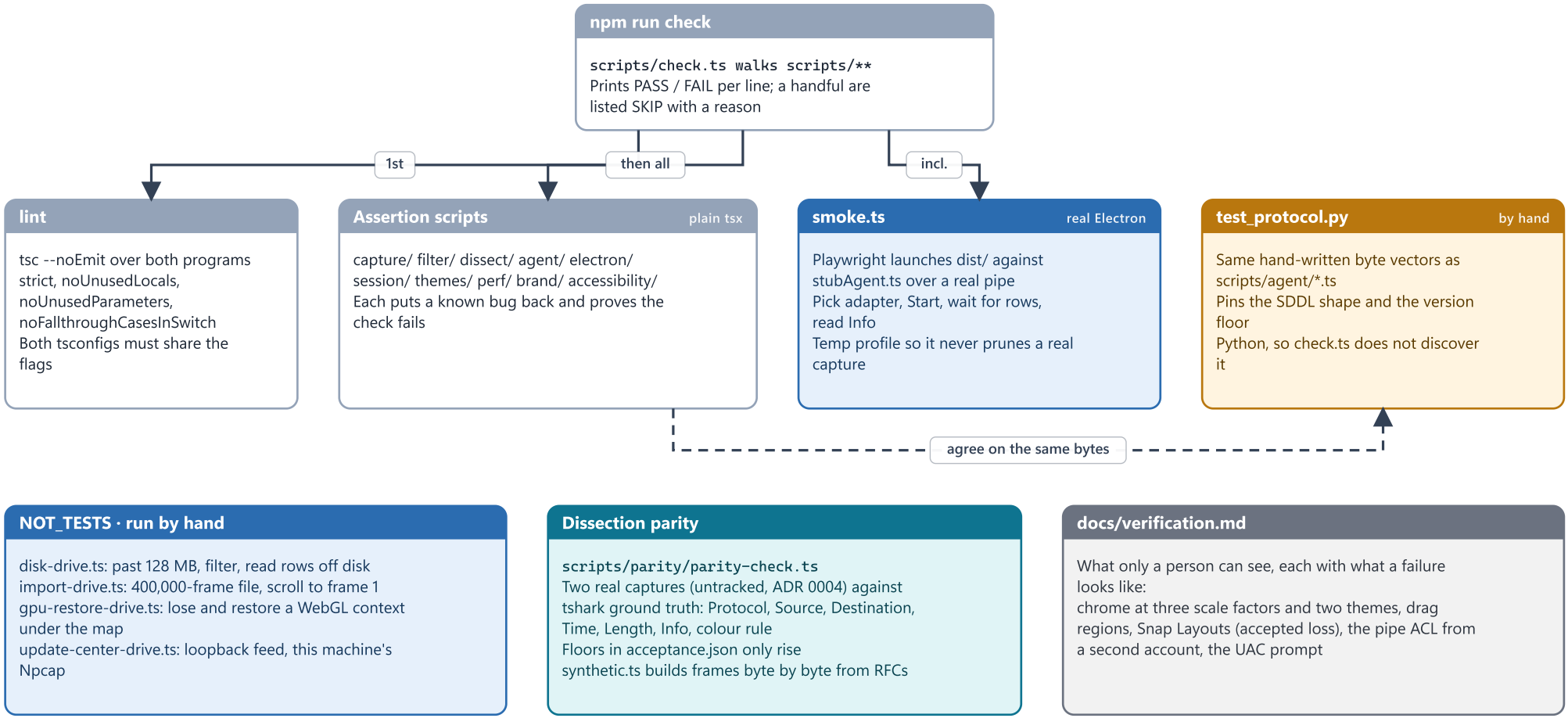


Every host this app can reach, and its gate — electron/csp.ts · OUTBOUND_HOSTS + MAIN_OUTBOUND_HOSTS

Renderer, in the CSP: get.geojs.io (map geolocation) and ipapi.co (public identity) → behind nameResolution.geoIp · cloudflare-dns.com (DoH reverse lookup) → behind nameResolution.reverseDns · tiles.openfreemap.org (basemap) → no switch, loads whenever the map is shown; the offline d3-geo map is the open item
Main, not in the CSP: updates.inwire.app (feed) → inwire.updates, off until asked · npcap.com (driver download) → behind a click. Nothing else. No account, no telemetry, no backend (product invariant: offline by default).

Privilege falls from left to right. The agent runs as LocalSystem because that is what lets Npcap be installed admin-only and lets a capture outlive the window; its pipe's ACL is the whole authentication (ADR 0029). Main is the only unsandboxed Node and never parses a frame. The store touches the file system and nothing else. The renderer has no Node at all, and the Worker is the one component that reads hostile bytes: bytes in, summaries out. Every outbound host is listed once, with its consent gate.

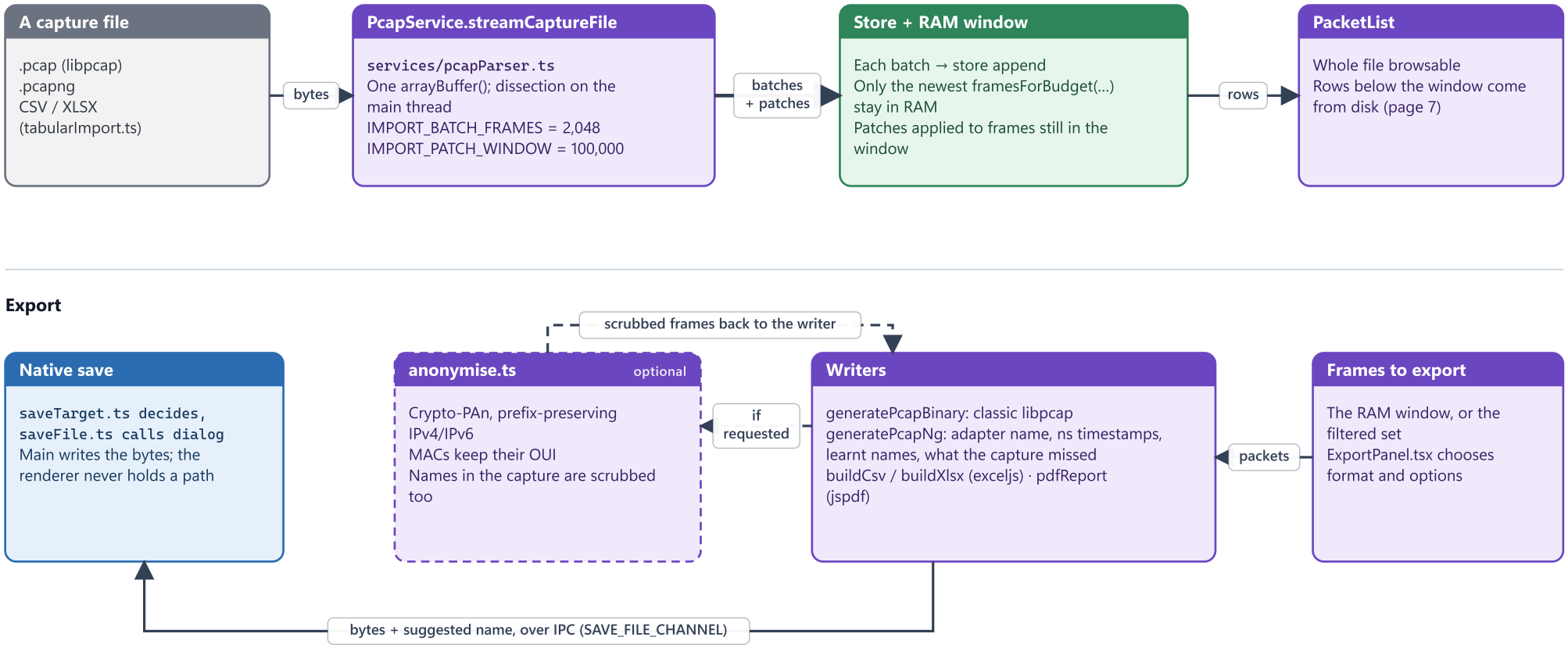
Plain tsx for everything that can be, a real Electron for what cannot



The pattern that keeps the suite Electron-free: pipeTransport.ts beside agentPipe.ts, saveTarget.ts beside saveFile.ts, agentHealth.ts beside agentService.ts. The decision is importable; the Electron call is not.

npm run check type-checks both TypeScript programs and then runs every assertion script under scripts/ with plain tsx: no Electron, no browser, so the suite runs in CI on Linux and on every commit. That property is protected by splitting decisions away from the modules that need Electron. Three classes of bug were invisible to all of that and were found by hand, so smoke.ts launches the real app over a real named pipe against a stub agent and reads rows off the screen. The slow drives and the manual list cover what even that cannot see.

Import streams to disk, export goes through a native dialog



Top: an imported file is read whole but dissected in batches of 2,048 frames, each written to the store as it is produced, so a 400,000-frame file imports completely where it used to stop at 181,000. Dissection amends frames it has already emitted, so frames stay reachable for a patch for 100,000 frames, the same rule the writer follows. Bottom: export walks the frames in the RAM window, optionally anonymised, and hands bytes to main, which owns the save dialog and the file handle because the renderer has neither.

The map from concept to file

local-agent/

agent.py the service loop, hello, capture
pipe_server.py the pipe + SDDL
pipe_frame.py u32 len + u8 kind
packet_batch.py IWP1 + 18-byte records
npcap_access.py reads the device DACL
service_windows.py SCM integration
test_protocol.py byte vectors, floors

electron/

main.ts the adapter
window.ts · windowState.ts · windowColors.ts
csp.ts hosts and the policy
agentPipe.ts · pipeTransport.ts
agentService.ts service diagnosis
preload.ts · channels.ts the bridge
store/ host · entry · volume · heapGuard
updateCenter/ manifest · verify · elevate · npcap

src/services/ (engine)

pcapParser.ts front door
dissectors/ one file per protocol
dissect/ fieldRegistry, fieldBlock, tcpAnalysis, orderingWindow, tokenPool
pipeline/replay.ts trees on demand
filter/ lexer · parser · compile · fields
tls/ keyLog · decrypt
*Tracker.ts TCP, DNS, QUIC, ICMP, L3

src/services/capture/

captureHost.ts message union
captureWriter.ts · captureReader.ts
chunkCodec.ts · captureIndex.ts
captureScan.ts · matchSet.ts
blockFile.ts → nodeBlockFile.ts
storePacket.ts the boundary flatten
captureDirectory.ts planPrune
serialQueue.ts

src/ (UI)

App.tsx controller with a view
hooks/ usePacketBuffer, useAgentConnection, useKeyboardNavigation
services/packetBuffer.ts prepare / commit
services/bufferBudget.ts
lib/rowSource.ts · captureRowSpace.ts
components/common/ TitleRow, ActivityRail, FilterBar, Settings
components/packet-list · packet-details · status · statistics
services/viewPrefs.ts views, back steps

Everything else

scripts/ the assertion suite, check.ts
scripts/electron/smoke.ts the real app
installer/setup.iss the one installer
electron-builder.yml the pack
docs/decisions/ ADR 0001–0033
docs/verification.md the manual list
test/fixtures/ untracked captures
CLAUDE.md the long version

One card per area of the repository, listing the files that hold the decisions. CLAUDE.md is the long-form version of this document; docs/decisions/ holds the 33 ADRs behind it.